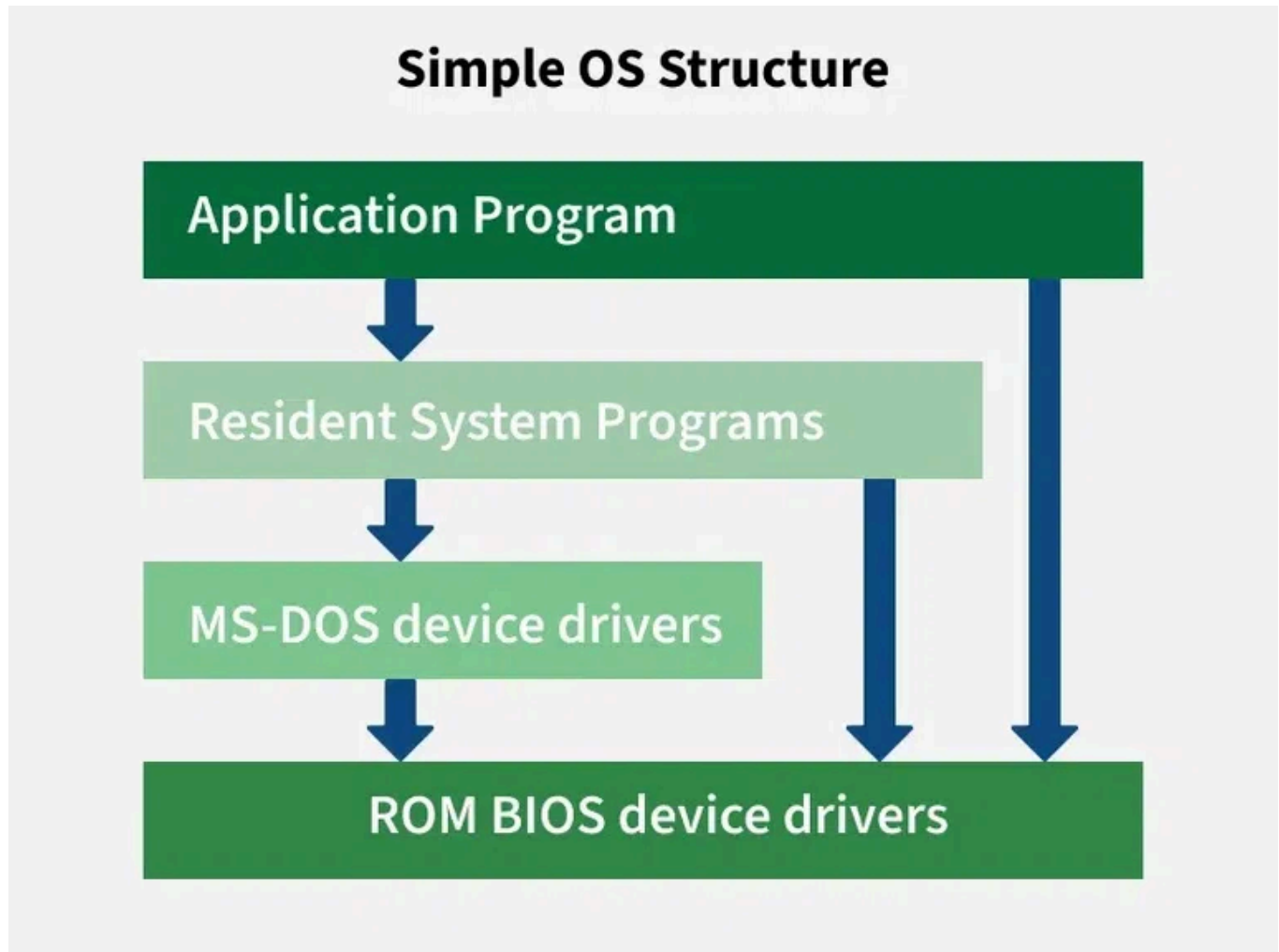
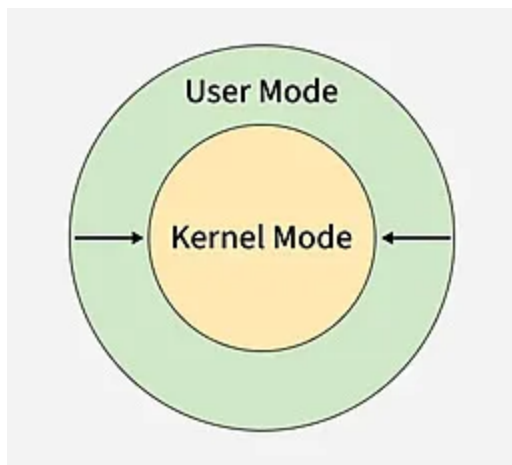
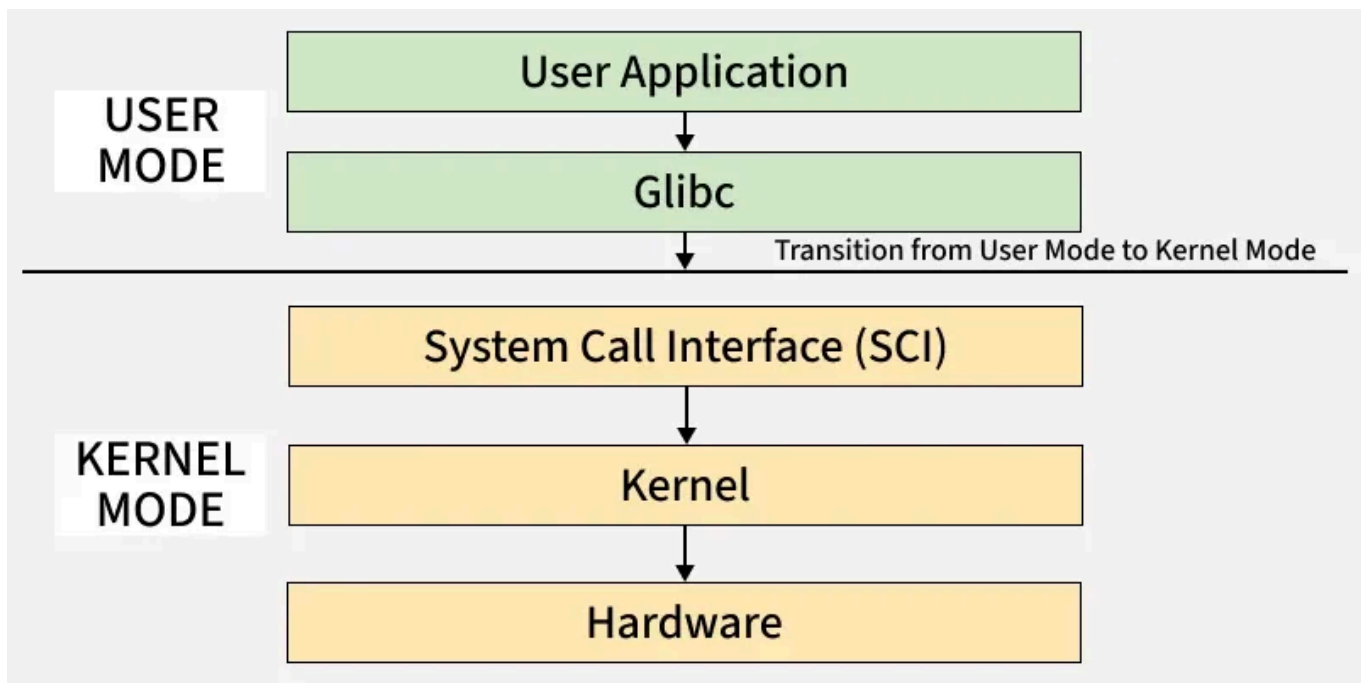


Lesson 2 - OS architectures

Simple OS Architecture



User Mode vs Kernel Mode



User Mode

- User Mode is a restricted environment where application programs run. When a program starts, the OS creates a separate process and assigns it its own memory space.
- Programs in user mode can't access hardware or kernel memory directly. They must request access through system calls to the kernel.

Key Features of User Mode

- **Isolated Environment***: Each process runs in its own memory space.
- **No Direct Hardware Access***: Only accessible through kernel services.

- **Safe Execution***: Errors or crashes don't affect the whole system.*

Advantages

- **Easier Debugging***: Bugs are contained to individual applications.
- **Crash Containment**: Only the crashing application is affected.
- **Process Isolation***: One app can't read or corrupt another's memory.
- **Controlled Access***: Kernel checks and authorizes requests.
- **Stability and Reliability***: Applications can't directly affect the kernel or other processes, reducing the risk of full-system crashes.
- ***Security**: Direct access to critical resources is blocked, preventing unauthorized use.*

Disadvantages

- **Performance Overhead**: System calls require switching between user and kernel mode, adding latency.
- **Limited Capability**: Low-level tasks (like direct I/O) cannot be done.
- ***System Call Latency****: Frequent transitions to kernel mode can slow performance.
- **Dependence on Kernel**: User programs rely on the kernel to perform essential functions.
- **Reduced Real-Time Use**: Less suitable for systems needing low-latency execution.
- **Less Developer Control**: Access to memory and execution is restricted.*

Kernel Mode

- Kernel Mode is the ****privileged mode**** where the core part of the operating system, the ****kernel**** executes. It has ****unrestricted access**** to all machine resources including CPU, memory, storage, and connected devices.
- When a program running in user mode makes a request that requires hardware access, the system switches the CPU into ****kernel mode**** to

perform the task. After execution, control returns to user mode.

Key Features of Kernel Mode

- **Direct Hardware Interaction:** Can communicate with I/O devices, memory, and processor.
- **Complete System Control:** Manages process scheduling, memory, and device drivers.
- **Privileged Instructions:** Executes instructions that affect system-wide behavior.

Advantages

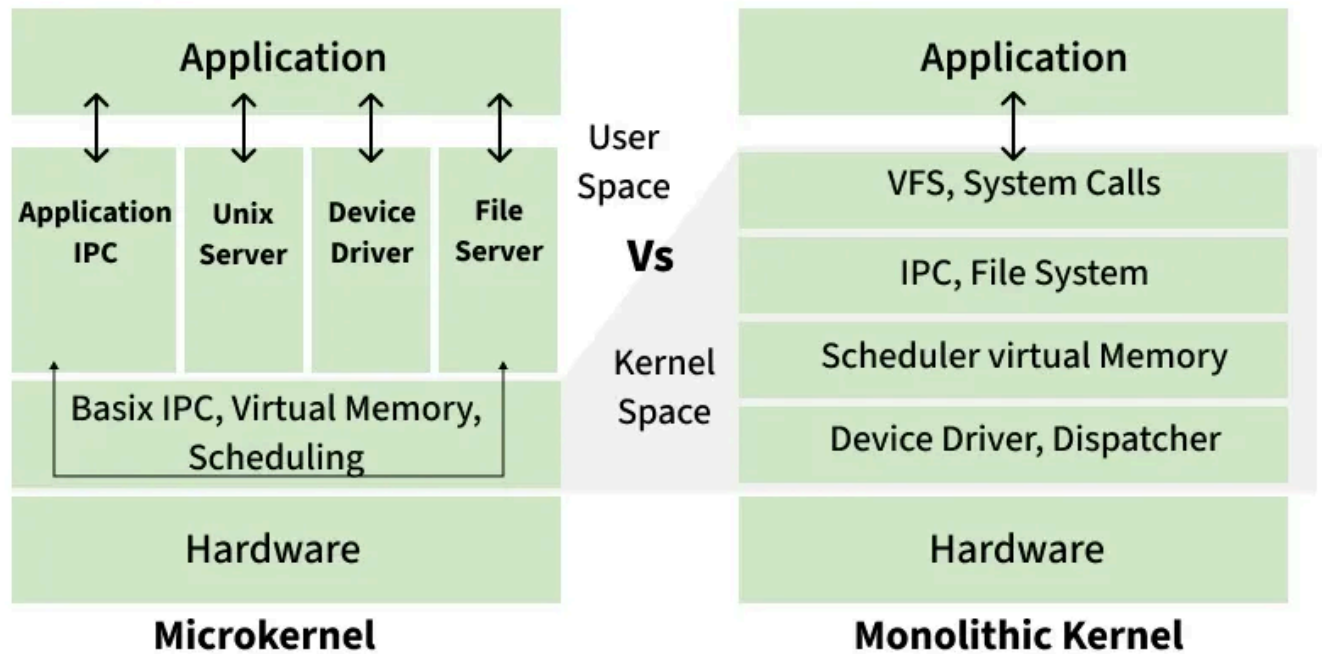
- **Full Hardware Access:** Required for tasks like device management and memory allocation.
- **Efficient Resource Management:** Can directly manage CPU time, RAM, and I/O devices.
- **Custom Optimization:** OS developers can fine-tune for performance or hardware.
- **Low Overhead:** Handles system calls and process management efficiently.
- **Advanced Scheduling:** Controls multitasking, thread synchronization, and context switches.

Disadvantages

- **High Risk of System Crash:** Errors or bugs can bring down the whole OS.
- **Harder to Debug:** Mistakes in kernel code are difficult to trace and fix.
- **Security Vulnerabilities:** A single faulty kernel driver can compromise the system.
- **Complex Development:** Requires deep knowledge of OS internals.
- **No Process Isolation:** Memory is shared; bugs can corrupt the entire kernel.

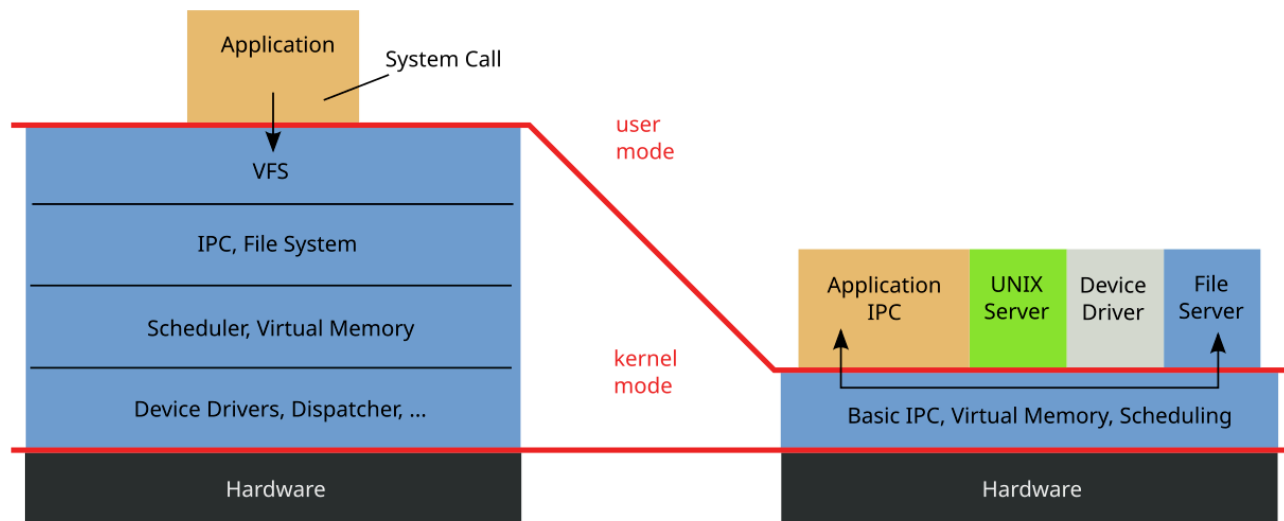
Kernel Mode	User Mode
The program has direct and unrestricted access to system resources.	The application program do not have direct access to system resources. In order to access the resources, a system call must be made.
The whole operating system might go down if any faults or errors occurs.	A single process fails if an interrupt occurs.
Kernel mode is also known as the master mode, privileged mode or system mode.	User mode is also known as the unprivileged mode, restricted mode.
All processes share a single virtual address space.	All processes get separate virtual address space.
The applications have more privileges as compared to user mode.	While in user mode the applications have fewer privileges.
As kernel mode can access both the user programs as well as the kernel programs there are no restrictions.	While user mode needs to access kernel programs as it cannot directly access them.
The mode bit of kernel-mode is 0.	While the mode bit of user-mode is 1.
It is capable of referencing both memory areas.	It can only make references to memory allocated for user mode.
A system crash in kernel mode is severe and makes things more complicated.	A system crash can be recovered by simply resuming the session.
Only essential functionality is permitted to operate in this mode.	User programs can access and execute in this mode for a given system.
The kernel mode can refer to any memory block in the system and can also direct the CPU for the execution of an instruction, making it a very potent and significant mode.	The user mode is a standard and typical viewing mode, which implies that information cannot be executed on its own or reference any memory block; it needs an Application Protocol Interface (API) to achieve these things.

Microkernel and Monolithic Kernel



Monolithic Kernel
based Operating System

Microkernel
based Operating System



Micro-Kernel

- A **microkernel** organizes the operating system by keeping only the most essential functions inside the kernel. These include basic process management, communication between system components, and minimal hardware control. All other services are placed outside the kernel in separate user-space programs.

- Components such as device drivers, file systems, and network services run as independent processes in user space.
- This approach keeps the kernel small, reduces the chance of system-wide failures, and makes the operating system more secure and easier to maintain.

Advantages of Micro-Kernel

- Size of Micro-Kernel is smaller, so it is easy to use.
- Easy to extend Micro-Kernel
- Easy to port Micro-Kernel
- Micro-Kernel is less prone to errors and bugs. One such example of this is Mac OS.

Disadvantages of Micro-Kernel

- The execution of Micro-Kernel is slower.
- Only the most important services are present inside the kernel and rest of the operating systems are present inside system application program.
- The communication between client process & services running in user address space is established through message that further reduces the speed of execution.

Monolithic Kernel

- A **monolithic** kernel design places all operating system components—both user services and kernel-level services—into the same address space, running entirely in kernel mode. This arrangement allows different parts of the system to interact quickly and efficiently without additional communication overhead.
- This architecture includes major functions such as memory management, process control, device drivers, and file systems within one large kernel.
- While this setup can improve performance, it also increases complexity, as a problem in any single component can affect the entire system and

make maintenance more challenging.

Advantages of Monolithic Kernel

- Monolithic Kernel is an all in one piece where user services & kernel services are implemented under the same address space.
- It has a faster speed of execution.
- One such example of this is Linux. It is simple to design and has a very high performance.

Disadvantages of Monolithic Kernel

- The monolithic kernel has a larger size since both user services & kernel services are implemented under the same space.
- Since it is larger in size, it becomes hard to extend the Monolithic Kernel.
- Hard to export and port the monolithic kernel
- Unlike micro kernel, Monolithic kernel is more prone to errors and bugs. Thus, the system encounters more errors than the usual being.

Microkernel vs Monolithic Kernel

Microkernel	Monolithic kernel
User services and kernel services are placed in separate address spaces.	User and kernel services share the same address space.
More complex to design.	Easier to design and implement.
Smaller in size.	Larger compared to microkernel.
New functionalities can be added more easily.	Adding new functionalities is more difficult.
Requires more code to design.	Requires less code than a microkernel.
Failure of one component does not affect the entire system.	Failure of one component can cause the whole system to fail.
Slower execution speed.	Faster execution speed.
Easier to extend.	Not easy to extend.
IPC uses messaging queues.	IPC uses signals and sockets.
Easier to debug.	More difficult to debug.

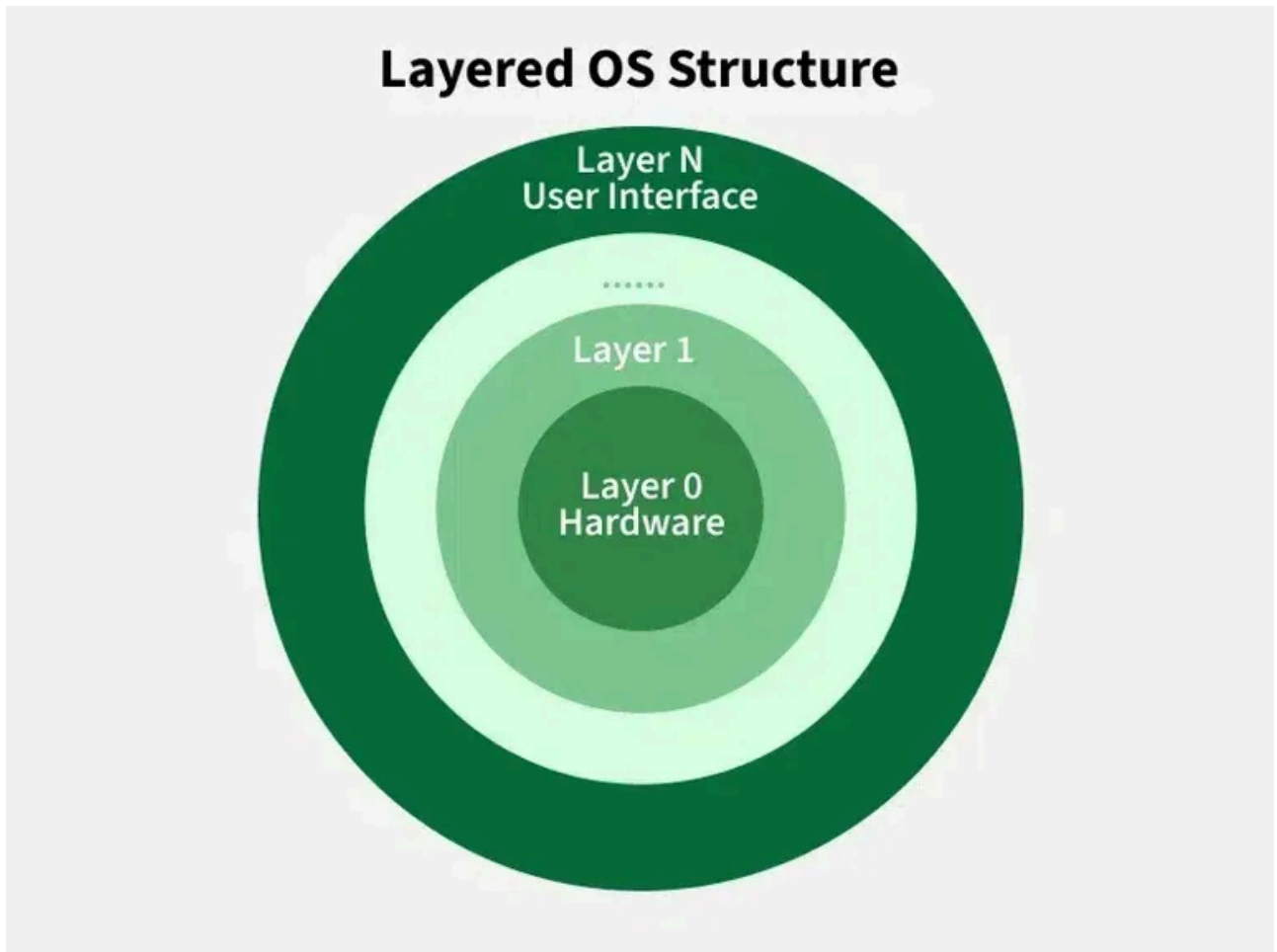
Microkernel	Monolithic kernel
Simple to maintain.	Requires extra time and resources for maintenance.
Requires message forwarding and context switching.	Does not require message passing or context switching during normal operations.
Kernel provides only IPC and low-level device management.	Kernel includes all major OS services.
Example: macOS.	Example: Windows 95.

Layered Structure

In **Layered structure**, the OS is divided into levels where **each layer uses the services of the one below it**—**hardware** at the **bottom** and the **user interface at the top**. This design improves organization and **simplifies debugging**, as **errors can be isolated to specific layers**. However, it adds overhead because data must pass through multiple layers and requires careful planning. **UNIX** is an example of this structure.

- OS is divided into multiple layers (hardware at bottom, UI at top).
- Each layer uses the functions of the layer below it.
- Easier debugging because errors are isolated to specific layers.
- Adds overhead due to data passing through many layers.
- Requires careful planning; layers can only use lower layers.

- UNIX is an example of a layered structure.



Hybrid OS Architecture

